

Document number: P2038r0
Date: 2020-01-11
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>
Ryan McDougall <mcdougall dot ryan at gmail dot com>

Proposed nomenclature for contract-related proposals

This document proposes a number of terms that we recommend to authors of contract-related proposals, and SG21 members for use, in order to avoid misunderstandings. The terms have been chosen so that:

- Names that have overloaded meaning, are ambiguous or misleading are avoided; in particular:
 - "assume" — as it is not clear who assumes: it can have meaning of `__builtin_assume()` or it can have meaning as in, "function *assumes* that its precondition holds".
 - "expect" — as it seems only to express an emotional state; is not clear what is the relevance of these expectations. You can "expect" that people will use your function incorrectly.
 - "assert" — as "assert" could mean "trust me, I say this and this is true", or "test this hypothesis for me". In fact, clang-tidy interprets `assert()` in the former way: if it can see expression `e` to be asserted, it does not try to verify if it is true: instead, it just trusts the programmer.
 - "check" (used alone) / "contract check" — as it implies that some run-time checking will be performed or required, whereas some predicates are not even expressible as code that could be executed.
 - "contract" (used alone) — as this is something between the humans: not something that function declarations or the implementation would ever be affected with.
- The focus is shifted from generating additional code for run-time checking by the compiler to declaring information consumable by other tools that the Standard refers to as "implementation", such as static analyzers or sanitizers.

Recommended terms

Contract annotation

It has a *location* in code: e.g., just before the function execution begins, or just after the function execution end successfully. It has a *predicate*, such as `p != nullptr`. Its "semantics" is: if the execution reaches the location of contract annotation, and the predicate can be determined (not necessarily by evaluating it) to be `false`, then the program has a *contract violation*. The information about a contract violation in the program can be used in different ways by different tools, including the compiler.

Thus, contract annotation helps provide a definition of a bug that is understandable by machines. Because contract violation is for sure a manifestation of a bug.

Rationale: A longer term "contract annotation" is recommended in order to avoid shorter "contract" which can also mean "everything that function guarantees and expects, including runtime performance guarantees." Also, a "contract" is a pair: a precondition and a postcondition, such as "the output is 2 provided that the input is 1", and statement "the output is 2" alone is not a contract.

Contract violation

A contract violation results from an interaction between the code before the contract annotation and the contract annotation itself. The behavior of a program with a contract violation can differ from one translation to the other based on compiler switches. This is similar to implementation-defined behavior except that we try to specify in more detail what the variance in behavior is, and under what conditions.

A program with a contract violation can still be a well-formed, UB-free program. Unless other UB kicks in later on, you can still reason about the behavior of the program in terms of operations on the abstract machine.

Contract violation is a symptom of a bug located either before the contract annotation or inside its predicate. Contract annotations can detect symptoms of bugs, but not bugs themselves. Implementations are encouraged to detect and prevent contract violations by different means. Although there may be reasons for intentional contract violations (e.g., in unit tests), it is generally agreed that the information about these violations from static analyzers is welcome, and never treated as "false positives".

Rationale: speaking in terms of "contract violation" (or the lack thereof) avoids using word "expect" as in "a precondition is 'expected' to hold", where it is not clear if an "expectation" has any formal meaning or consequences. Term "library UB" is also not encouraged as there might be no library in sight when a programmer puts a contract annotation in her function.

Fact

An information that an implementation has about the state of variables at the given point of program execution. A fact can be used for different things by the implementation, including code transformations. E.g.,

```
int normalize(int i)
{
    if (i > 0) return 1;
    if (i < 0) return -1;

    // at this point it is a fact that `i == 0`
    return i; // can be replaced with `return 0;`
}
```

Derived fact

A fact, like the one illustrated above, that the implementation obtains by means other than direct input from the programmer. A fact derived from other facts is a derived fact.

UB-derived fact

A derived fact obtained through special properties of undefined behavior; e.g.,

```
int g(int i)
{
    if (i != 0)
        *((void*)0) = 0; // UB

    // at this point it is a (UB-derived) fact that `i == 0`
    return i; // can be replaced with `return 0;`
}
```

Injected fact

An injected fact is a fact that an implementation cannot derive by other means than by direct input from the programmer through a dedicated instruction.

```
int f(int i)
{
    __builtin_assume(i == 0); // Compiler is allowed to apply code transformations as if `i == 0` were true.

    // at this point it is an (injected) fact that `i == 0`
    return i; // can be replaced with `return 0;`
}
```

Rationale: The usage of somewhat artificial terms "injected/derived fact" is recommended in order to avoid terms "assume" and "assert" that have multiple meanings in this context.

Run-time check

From a contract annotation in location L with predicate P , an implementation can, under implementation-defined conditions (but we try to specify the conditions with "levels", "build modes" and other "toggles"), insert at location L a piece of code that is executed at run-time, possibly with side effects, of the form:

```
{
    if (!P) HANDLE_CONTRACT_VIOLATION();
}
```

This is called a *run-time check*. `HANDLE_CONTRACT_VIOLATION()` is subject to further implementation-defined behavior, such as calling user-provided function, or doing nothing, or maybe doing something even different.

This definition leaves open the question whether or not P should be allowed to have side effects, as well as what "side effects" would mean in this context.

When a run-time check is inserted from a contract annotation with predicate P , and `HANDLE_CONTRACT_VIOLATION()` is a call to a function that never returns normally, then P is a derived fact after the run-time check.

Rationale: using a compound "run-time check" as something produced from "contract annotation" makes it clear that the two are different things. A run-time check is only one of a number of uses of contract annotations.

Function contract

A *function contract* of function f is an information associated with f . This information is not necessarily consumable by automated tools or machines. This is an information meant for humans, either provided in the form of documentation or comments in the declaration of f , necessary to be able to correctly call and implement f . This is a means of communication between the implementer and the user of f .

A function contract consists of two parts indicating (1) what the function guarantees, and (2) under what conditions. The things that a function can guarantee include:

- that certain values are never returned (function's range),
- the relation of output values to the input values (e.g., that the result will be the sum of the input parameters),
- runtime complexity,
- how function failures are communicated,
- level of thread safety,
- level of exception safety.

The conditions that the function contract of function f can include:

- what input values are not allowed (function's domain),
- up to how many times can f be called,
- what functions have to be called prior to calling f .

While this is not a complete definition of "function contract", it reflects its relevant properties:

1. Function contract cannot be fully expressed in C++, even if we add "contract support" to the language.

2. Parts of function contract can be expressed using different features of c++:
 - annotations on functions such as `noexcept` or `[[noreturn]]`,
 - strong types, such as `Range<int>` instead of `std::pair<int, int>`,
 - precondition and postcondition contract annotations.
3. Contract annotations can only express a subset of what function contract guarantees or under what conditions the guarantees are provided.
4. Function contract does not describe full semantics of a function, and a function can be called with its contract violated and produce the results desired by the programmer, even if these results are not guaranteed in the contract. For instance, function `f`'s contract can say that it is a contract violation to pass a null pointer as an input. However, the implementer of `f` can nonetheless runtime-check if the input pointer is null and throw an exception in this case. The caller may have learned about this by means other than function contract: e.g., by looking at the implementation, and gained confidence that in this release of the library this will throw, even if not guaranteed in the contract.

Rationale: Using a longer term "function contract" makes it clear that we do not mean a single precondition/postcondition.

Acknowledgements

The distinction between the contract as a whole and individual preconditions and postconditions was taken from [\[P1332r0\]](#). The unambiguous terms "injected fact" and "derived fact" have been suggested by Herb Sutter. Joshua Berne and Hyman Rosen helped shape the definitions.

References

1. [P0542r5] -- G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/P0542r5.html>).
2. [P1332r0] -- Joshua Berne, Nathan Burgers, Hyman Rosen, John Lakos, "Contract Checking in C++: A (long-term) Road Map" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1332r0.txt>).